`tests/letsencrypt/cert_analyzer.sh` — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:** Active sourceable library of PURE, hermetic, offline cert-chain / expiry analyzer functions for the Let's Encrypt Phase-3 workstream.

> Companion (§11.4.18) to the in-source documentation block at the top of the script. This is a **sourced library**, not a runnable script — source it and call its functions.

Overview / Purpose

A sourceable library of PURE functions over a PEM certificate / chain file. It answers the Let's Encrypt workstream's Phase-3 questions about a RESULTING certificate — is it valid now, how many days remain, does its SAN cover the hostname, was it issued by the EXPECTED CA, is renewal due — WITHOUT any network, ACME protocol, or container boot. It validates a cert regardless of HOW it was obtained (Caddy / lego / certbot / Pebble / LE staging / LE prod), so it is client-and-challenge-AGNOSTIC and independent of the pending operator decisions (design §9: DNS-01 vs HTTP-01, Caddy vs lego).

Every time-aware function is deterministic under an injected "now" epoch (§11.4.50), so a single fixture can be tested as valid AND as expired with no time-travel. The library is self-validated against golden-good + golden-bad fixtures by `cert_analyzer_selftest.sh` (§11.4.107(10)).

Public function contract

Function	Returns
`cert_not_expired <pem> [now]`	`0` iff `NotBefore <= now < NotAfter`; non-zero if expired / not-yet-valid; `2` if unparseable.
`cert_days_remaining <pem> [now]`	prints whole days from `now` to `NotAfter` (may be negative); non-zero (no output) on parse failure.

Function	Returns
`cert_san_matches <pem> <host>`	`0` iff `<host>` matches a SAN dNSName exactly OR via a single leading-wildcard; `1` no match; `2` empty host / parse failure. Never a substring match; no CN fallback.
`cert_chain_roots_in <leaf> <ca>`	`0` iff `<leaf>` verifies to `<ca>` (issued by expected CA), validity-period-independent (`-no_check_time`); `2` if a file is missing.
`cert_renewal_due <pem> <threshold_days> [now]`	`0` (DUE) iff days-remaining `<= threshold`; non-zero if not due; `2` on parse failure.

Internal helpers `_cert_now` and `_cert_epoch` are not part of the public contract.

Usage

```
. tests/letsencrypt/cert_analyzer.sh      # source it; do NOT
        execute

cert_not_expired    good_leaf.pem "$now_epoch"
cert_days_remaining good_leaf.pem "$now_epoch"
cert_san_matches    good_leaf.pem proxy.test
cert_chain_roots_in leaf.pem      test_ca.pem
cert_renewal_due    leaf.pem 30   "$now_epoch"
```

Deterministic "now" seam (§11.4.50)

Every time-aware function takes an OPTIONAL trailing `<now_epoch>` (unix seconds, UTC). Precedence, highest first: (1) the explicit positional argument; (2) the `CERT_ANALYZER_NOW_EPOCH` env var (per-run seam); (3) the real wall clock (`date -u +%s`).

Inputs

- PEM certificate / chain files (public material only — see the §11.4.10 note below). Parsing is entirely via `openssl x509` / `openssl verify`.
- Optional `<now_epoch>` argument or `CERT_ANALYZER_NOW_EPOCH`.

Outputs

- Return code only for the boolean functions (`0` = property holds; non-zero = does not hold or unparseable).

- `cert_days_remaining` additionally prints a signed integer day count on stdout.

Side-effects

None — pure. It only READS the PEM file, never writes, never networks.

Dependencies

POSIX `sh`, `openssl` (`x509` + `verify`), GNU `date -d` (epoch parse), `awk`/`sed`/`tr`. POSIX-clean — parses under `sh -n` AND `bash -n` (§11.4.67); no bash-only constructs (`[[ ]]`, `<<<`, arrays, `>( )`, `${v^^}`, `local`).

§11.4.10 note — no real secrets

The library only READS certificate (public) material. Its fixtures are generated by `tests/letsencrypt/fixtures/gen_fixtures.sh` from EPHEMERAL throwaway self-signed test CAs — NOT real LE certs, NO production private keys. The generated `*.key` / `*.csr` / `*.srl` are gitignored (regenerated per §11.4.77); only the public `*.pem` certs are tracked.

Edge cases

- **Unparseable / absent / truncated PEM** → boolean functions return `2` (no false-PASS, §11.4.1).
- **Expired cert that roots correctly** → PASSes `cert_chain_roots_in` (issuance is time-independent) while FAILing `cert_not_expired` — the two checks are orthogonal.
- **`*.example.com`** covers exactly one extra left label (`a.example.com`) but NOT the apex `example.com` and NOT `a.b.example.com`.
- **Empty-SAN cert** never matches a hostname (no CN fallback); **IP-only SAN** never matches a DNS name (dNSName-only discipline).
- Boundary semantics: `NotBefore` inclusive, `NotAfter` exclusive.

Related scripts

- `tests/letsencrypt/cert_analyzer_selftest.sh` — TAP self-test that self- validates every function against golden-good / golden-bad fixtures.
- `tests/letsencrypt/fixtures/gen_fixtures.sh` — the fixture generator.

- `tests/lib/evidence.sh` — the sibling data-plane evidence library whose `EVIDENCE_*` override style the now-seam mirrors.
- Design `docs/design/LETSENCRYPT_HTTPS.md` §6 + `LETSENCRYPT_HTTPS_PLAN.md` Phase 3; Constitution §11.4.107(10) / §11.4.50 / §11.4.10 / §11.4.135.

Last verified

2026-07-01 — documented against the library source; `sh -n` / `bash -n` parse-clean. Correctness proven hermetically (no network, no ACME) by `cert_analyzer_selftest.sh`.